Guía paso a paso — Clase 4: Red neuronal simple (parte 2)

Temas: medir el error · regla delta (descenso por gradiente en una neurona) · entrenamiento completo · inicialización de pesos · ADALINE · red con varias salidas
Duración: ~2 h 30 min
Material: diapositivas `Clase4_Diapositivas` (PDF y PowerPoint, 21 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
Requisito: Clase 3 (funciones de activación, derivada y descenso por gradiente en una variable)
Lecturas base (libro del Dr. Irving Vasquez):

- Descenso por gradiente (aprendizaje)
- Redes neuronales de varias salidas

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Medir el error de una red con el error cuadrático medio (MSE).
2. Aplicar a mano un paso de la **regla delta**: término de error, incremento y actualización.
3. Entrenar una neurona completa con descenso por gradiente y medir su exactitud.
4. Explicar por qué los pesos se inicializan al azar y cómo funciona la inicialización de Xavier.
5. Calcular la salida de una red de una capa con varias salidas usando matrices.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría explicada desde cero, con ejemplos resueltos.
- **Parte C:** práctica en la computadora, paso a paso.
- **Parte D:** problemas frecuentes.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación, objetivos y repaso de la Clase 3 (10 min)	1–4
0:10	B1. Medir el error (15 min)	5–6
0:25	B2. La regla delta (25 min)	7–10
0:50	C1. Práctica: un paso del descenso (20 min)	11
1:10	Descanso (10 min)	12
1:20	B3. Entrenamiento completo e inicialización (15 min)	13–15
1:35	C2. Práctica: entrenar una neurona (25 min)	16

Tiempo	Tema	Diapositivas
2:00	B4. ADALINE (8 min)	17
2:08	B5. Red con varias salidas (12 min)	18–19
2:20	Quiz y cierre (10 min)	20–21

PARTE A — Antes de clase (15 min)

A.1 Actualizar el repositorio y probar la práctica

En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

```
conda activate nni
```

```
python docs/practica_04.py
```

Verificación: imprime cuatro bloques (regla delta, OR y XOR, ADALINE, red de varias salidas) y al final `Todas las comprobaciones pasaron.`

Para ver las gráficas (la pérdida bajando y la recta de ADALINE):

```
python docs/practica_04.py --graficar
```

A.2 Checklist

- [ ] `python docs/practica_04.py` funciona.
- [ ] VS Code abre `notebooks/1_07_descenso_por_gradiente.ipynb` con el kernel **nni** y la **primera celda corre sin error** (carga `data.csv` con `data_prep`).
- [ ] En el pizarrón: una "parábola del error" con el peso w en el eje horizontal y el error E en el vertical.
- [ ] Calculadora con e^x.

PARTE B — Teoría desde cero

Repaso de la Clase 3 (5 min) — diapositivas 1–4

Pregunta a la clase:

- "¿Qué hace la sigmoide?" → convierte h en un número entre 0 y 1, de forma suave.
- "¿Qué es la derivada?" → la pendiente: hacia dónde y cuánto sube la función.
- "¿Cuál es la regla del descenso?" → $x \leftarrow x - \eta \cdot f'(x)$.

Hoy: en lugar de bajar por x^2, vamos a bajar por **el error de la red**, y lo que moveremos serán **los pesos**. Eso es **entrenar**.

B1. Medir el error (15 min) — diapositivas 5–6

Para mejorar, la red primero necesita **un número que diga qué tan mal lo está haciendo**. A ese número se le llama **error**, **pérdida** o **función de costo** (*loss*); son sinónimos en este curso.

Tres formas de medirlo

Nombre	Fórmula	Comentario
Error residual (de un ejemplo)	$e = y - \hat{y}$	Puede ser positivo o negativo
Suma de errores al cuadrado (SSE)	$\Sigma (y - \hat{y})^2$	Elevar al cuadrado evita que errores + y − se cancelen
Error cuadrático medio (MSE)	$(1/m) \Sigma (y - \hat{y})^2$	El promedio: no crece solo por tener más datos

(m = número de ejemplos.)

¿Por qué al cuadrado? Si un ejemplo se equivoca +1 y otro −1, la suma simple daría 0, como si no hubiera error. Al elevar al cuadrado, ambos cuentan. Además, castiga más los errores grandes.

Ejemplo (diapositiva 5): etiquetas y = [1, 0, 1], predicciones $\hat{y}$ = [0.8, 0.3, 0.6].

- Errores: 0.2, −0.3, 0.4 → cuadrados: 0.04, 0.09, 0.16.
- SSE = 0.29 · MSE = 0.29 / 3 ≈ **0.0967**.

La pérdida es una montaña (diapositiva 6)

Si dibujamos el error E en función de **un peso** w, sale una curva con forma de valle. **Entrenar = bajar a ese valle moviendo w.** Es exactamente lo de la Clase 3: el descenso por gradiente, pero ahora la variable es el peso.

B2. La regla delta (25 min) — diapositivas 7–10

La derivada del error respecto a un peso (diapositiva 7)

Para una neurona con un solo ejemplo usamos $E = ½ (y - \hat{y})^2$. El ½ solo está para que las cuentas queden limpias.

Cambiar un peso w_i provoca un **efecto dominó**:

> w_i cambia → **h** cambia → **$\hat{y}$ = f(h)** cambia → **el error E** cambia.

La derivada total es la multiplicación de los efectos de cada eslabón. A esto se le llama **regla de la cadena**:

Eslabón	Derivada
E respecto a $\hat{y}$	$-(y - \hat{y})$

Eslabón	Derivada
ŷ respecto a h	f′(h) (la pendiente de la activación)
h respecto a w_i	x_i (porque h = … + $w_i x_i$ + …)

$$\frac{\partial E}{\partial w_i} = -(y - \hat{y})\, f'(h)\, x_i$$

Aplicando el descenso (w ← w − η·derivada), los dos signos menos se cancelan:

$$w_i \leftarrow w_i + \eta\,(y - \hat{y})\, f'(h)\, x_i$$

La regla delta en tres pasos (diapositiva 8)

Para que sea fácil de calcular a mano, se separa en tres pasos:

Paso	Fórmula	En palabras
1. **Término de error**	δ = (y − ŷ)·f′(h)	Cuánto y hacia dónde corregir
2. **Incremento**	$\Delta w_i = \eta \cdot \delta \cdot x_i$ (y Δb = η·δ)	Cuánto mover cada peso
3. **Actualización**	$w_i \leftarrow w_i + \Delta w_i$ · b ← b + Δb	Mover los pesos

δ se lee "delta"; por eso se llama **regla delta** (Widrow y Hoff, 1960). Con la sigmoide, f′(h) = ŷ·(1 − ŷ), lo que vimos en la Clase 3.

Ejemplo resuelto (diapositiva 9) — los mismos números del notebook `1_06`

Datos: x = [1, 1], y = 1, w = [0.1, 0.2], b = 0, η = 2, activación sigmoide.

1. **Predicción:** h = 0.1·1 + 0.2·1 + 0 = 0.3 → ŷ = σ(0.3) ≈ **0.5744**
2. **Derivada de la sigmoide:** f′(h) = 0.5744·(1 − 0.5744) ≈ **0.2445**
3. **Término de error:** δ = (1 − 0.5744)·0.2445 ≈ **0.1040**
4. **Incremento:** Δw = 2·0.1040·[1, 1] ≈ **[0.2081, 0.2081]** · Δb ≈ **0.2081**
5. **Actualización:** w ≈ **[0.3081, 0.4081]** · b ≈ **0.2081**
6. **Comprobar:** h ≈ 0.924 → ŷ ≈ 0.7159. La pérdida ½(y − ŷ)² baja de **0.0905** a **0.0404** ✓

Comparación con el perceptrón (diapositiva 10)

	Perceptrón (Clase 2)	Regla delta (hoy)
Activación	Escalón	Continua (sigmoide, lineal…)
Corrección	η·(y − ŷ)·x	η·(y − ŷ)·**f′(h)**·x
De dónde sale	Intuición de Rosenblatt	Descenso por gradiente (matemática)
Qué indica el error	Solo si acertó o no	Qué tan lejos estuvo

¡Se parecen mucho! La regla delta añade f′(h), que viene de derivar.

B3. Entrenamiento completo e inicialización (15 min) — diapositivas 13–15

El algoritmo completo (diapositiva 13)

En la práctica, la red se entrena con **muchos ejemplos** y durante **muchas épocas** (pasadas por todo el conjunto de datos):

1. **Inicializar** los pesos (ver abajo).
2. Para cada **época**:
    1. Poner los incrementos acumulados en cero.
    2. Para cada ejemplo (x, y): calcular $\hat{y}$, el término de error δ y **sumar** el incremento $\delta \cdot x$ al acumulado.
    3. Actualizar los pesos con el **promedio** de los incrementos multiplicado por η (dividir entre el número de ejemplos).
    4. Medir el error (MSE) para ver si va bajando.
3. Al final, medir qué tan bien funciona con datos **que no se usaron para entrenar** (prueba).

A esta forma (actualizar una vez por época con el promedio) se le llama descenso por gradiente **por lotes**.

Inicialización de los pesos (diapositiva 14)

- **¿Por qué no empezar en cero?** Todas las predicciones saldrían iguales y, en redes más grandes, todas las neuronas aprenderían lo mismo. Se rompe la "simetría" empezando **al azar**.
- **¿Por qué valores pequeños?** Con pesos grandes, h es grande, la sigmoide **se satura** y su pendiente es casi 0: la red aprende muy lento (Clase 3).
- **Inicialización de Xavier:** valores al azar uniformes entre $-1/\sqrt{n}$ y $+1/\sqrt{n}$, donde n = número de entradas de la neurona.
    - Ejemplo: con n = 6 entradas, los pesos se eligen entre **−0.408 y +0.408**.

Exactitud y el conjunto de datos de la práctica (diapositiva 15)

- **Datos:** admisiones a un posgrado (`notebooks/data.csv`, 400 alumnos).
    - **Entradas:** puntaje GRE, promedio GPA y el prestigio de la universidad de origen (`rank`, convertido en 4 columnas de 0/1). En total, **6 entradas**.
    - **Salida:** `admit`, que vale 1 si fue admitido y 0 si no.
    - El archivo `data_prep.py` ya escala los datos y separa **90 % para entrenar** (360) y **10 % para probar** (40).
- **Exactitud** (*accuracy*) = aciertos / predicciones. La red dice "admitido" si $\hat{y} > 0.5$.

B4. ADALINE (8 min) — diapositiva 17

- Es la neurona de Widrow y Hoff con **activación lineal**: $\hat{y} = w \cdot x + b$. Sirve para **regresión**, es decir, predecir un número.
- Se entrena con el mismo descenso por gradiente. Como f(h) = h, su derivada es f′(h) = 1, y la regla queda: $w \leftarrow w + \eta \cdot (y - \hat{y}) \cdot x$.
- **Ejemplo** (notebook `1_08`): predecir el **salario** a partir de los **años de experiencia** (30 personas).
- **Resultado con los datos estandarizados** (`docs/practica_04.py`): salario ≈ **9 450 · años + 24 848**. Para 5 años de experiencia → ≈ **72 098**.

- **Lección práctica:** en el notebook los datos no están escalados (los salarios valen decenas de miles). Por eso se usa una tasa diminuta (0.0001) y el sesgo aprende muy lento: la recta se ve razonable, pero no es la óptima. **Escalar los datos** (restar la media y dividir entre la desviación) hace el entrenamiento mucho más fácil.

B5. Red de una capa con varias salidas (12 min) — diapositivas 18–19

La idea (diapositiva 18)

Varias neuronas comparten **las mismas entradas**, y cada una da **su propia salida**. Por ejemplo, con datos del clima predecir a la vez la probabilidad de lluvia, la temperatura y la humedad.

- Cada entrada x_i se conecta con cada salida $\hat{y}_j$ por un peso w_{ij}. A esto se le llama capa **completamente conectada** (*fully connected*).
- Con m entradas y n salidas, los pesos forman una **matriz W de m filas × n columnas**, y hay **un sesgo por salida**: B = [b_1, …, b_n].

$$H = X\,W + B \qquad \hat{Y} = f(H)$$

La activación se aplica a cada elemento de H por separado.

Ejemplo resuelto (diapositiva 19)

X = [8, 4], W = [[−0.1, 0.6], [−0.8, 0.3]], B = [−0.7, 0.3], activación sigmoide.

- **Salida 1:** h_1 = 8·(−0.1) + 4·(−0.8) − 0.7 = −0.8 − 3.2 − 0.7 = **−4.7** → $\hat{y}_1$ = σ(−4.7) ≈ **0.0090**
- **Salida 2:** h_2 = 8·0.6 + 4·0.3 + 0.3 = 4.8 + 1.2 + 0.3 = **6.3** → $\hat{y}_2$ = σ(6.3) ≈ **0.9982**

> ⚠️ **Nota para el docente:** el libro trae este mismo ejemplo con H = [−7.7, 6.3] y ŷ = [$4.52\cdot10^{-4}$, $9.81\cdot10^{-1}$]. Las cuentas correctas son las de arriba. Puedes usarlo como reto: "encuentren el error del libro".

Número de parámetros: (m + 1)·n. Aquí (2 + 1)·2 = **6** (4 pesos + 2 sesgos).

PARTE C — Práctica en la computadora

Preparación (todos):

- VS Code → **Open Folder** → `curso-redes-neuronales`.
- Kernel **nni** en cada notebook.
- Ejecutar las celdas **en orden** con **Shift + Enter**.

C1. Notebook `notebooks/1_06_descenso_unpaso.ipynb` (20 min, 3 puntos) — diapositiva 11

Qué hay que hacer: los tres pasos de la regla delta, cada uno en su función.

1. **TODO (1 pt):** `error_term(y, x, neurona)` → δ = (y − ŷ)·f′(h).

- Pista: obtén h con `neurona.combinacion_lineal(x)`, ŷ con `neurona.forward(x)` y f′ con `sigmoid_prime(h)`.

2. **TODO (1 pt):** `incremento(eta, error_term, X)` → η·δ·X.
3. **TODO (1 pt):** `actualizacion(neurona, incremento_W, incremento_b)` → suma los incrementos a `neurona.W` y `neurona.b`.
4. En la celda final, `salida` es la predicción de la neurona: `neurona.forward(x)`.

Verificación (los números de B2):

Valor	Debe dar
Salida	≈ 0.5744
Término de error	≈ 0.1040
Incremento	≈ [0.2081, 0.2081]
Pesos actualizados	≈ [0.3081, 0.4081]

Después, vuelve a hacer la predicción: debe dar ≈ 0.7159, más cerca de y = 1 (la pérdida bajó).

C2. Notebook `notebooks/1_07_descenso_por_gradiente.ipynb` (25 min, 3 puntos) — diapositiva 16

Qué hay que hacer:

1. **TODO (1 pt):** inicializar `weights` con Xavier.
   - Pista: `np.random.uniform(-1/np.sqrt(n_features), 1/np.sqrt(n_features), size=n_features)`.
2. Ejecuta la celda de exactitud **antes** de entrenar: con pesos al azar sale baja.
3. **TODO (1 pt):** completar el ciclo, siguiendo el algoritmo de B3:
   - `output` → la predicción de `nodo` para `x`.
   - `error_term` → (y − output) por la derivada de la sigmoide. Recuerda que f′ = output·(1 − output).
   - `incremento_w +=` → el término de error por `x`.
   - Al terminar los ejemplos de la época: `weights +=` la tasa por el incremento acumulado, **dividido entre** `n_records`.
4. Grafica la pérdida: la curva debe **bajar**.
5. **(1 pt):** prueba distintas cantidades de épocas hasta superar **0.74** de exactitud.

Resultados de referencia (semilla 42, `learnrate = 0.5`):

Épocas	Pérdida al final	Exactitud
10	≈ 0.225	≈ 0.60
100	≈ 0.205	≈ **0.775** ✓
1000	≈ 0.197	≈ 0.725

Pregunta a la clase: "¿Por qué con 1000 épocas la pérdida baja pero la exactitud empeora?" → El conjunto de prueba es pequeño (40 alumnos) y la red empieza a ajustarse demasiado a los datos de entrenamiento. Es una primera probada de **sobreajuste**, que veremos más adelante.

⚠️ **Detalle importante:** escribe `weights += ...` y **no** `weights = weights + ...`. La neurona `nodo` comparte el mismo arreglo de pesos; con `+=` lo modificas "en el lugar" y `nodo` ve el cambio. Con `weights = weights + ...` creas un arreglo nuevo, `nodo` se queda con los pesos viejos y **la pérdida no baja**.

C3. Notebook `notebooks/1_08_adaline.ipynb` (demostración, sin puntos)

No tiene `TODO`: ejecútalo completo en vivo y comenta cada gráfica:

1. Los datos: más experiencia, más salario.
2. La recta antes de entrenar: al azar, muy mala.
3. La "superficie del error": el valle en 3D que el descenso tiene que bajar.
4. La pérdida durante el entrenamiento: baja.
5. La recta final.

Compárala con el resultado de `docs/practica_04.py`, que escala los datos (B4).

C4. Demo: `docs/practica_04.py --graficar` (5 min)

- OR se aprende: la pérdida cae a casi 0.
- **XOR no:** la pérdida se queda en 0.25. Una sola neurona sigue sin poder con XOR, aunque ahora use sigmoide y descenso por gradiente. Para resolverlo hace falta **otra capa**, y eso es el próximo módulo.

Quiz y cierre (10 min) — diapositivas 20–21

Quiz (en papel, 5 min):

Pregunta	Respuesta
¿Por qué el error se eleva al cuadrado?	Para que los errores + y − no se cancelen, y castigar más los grandes
¿Cuáles son los tres pasos de la regla delta?	Término de error, incremento, actualización
¿Qué añade la regla delta respecto a la del perceptrón?	La derivada de la activación f′(h)
¿Por qué no se inicializan los pesos en cero?	Las salidas serían iguales; hay que romper la simetría
¿Cuántos parámetros tiene una capa con 3 entradas y 2 salidas?	(3 + 1)·2 = 8

Para llevar:

1. Entrenar = bajar por la montaña del error moviendo los pesos.
2. Regla delta: $\delta = (y - \hat{y}) \cdot f'(h)$, $\Delta w = \eta \cdot \delta \cdot x$.
3. Una sola capa sigue sin resolver XOR → **redes multicapa** (próximo módulo).

Tarea:

1. Terminar `1_06` y `1_07` (6 puntos en total).
2. En `docs/practica_04.py`, cambia `eta` de la función `entrenar` a 0.1 y a 10: ¿qué pasa con la pérdida de OR?

3. Leer del libro: Redes neuronales multicapa.

Rúbrica:

Notebook	Puntos
1_06 Descenso, un paso	3
1_07 Descenso completo	3
1_08 ADALINE	Demostración, sin puntos

PARTE D — Problemas frecuentes

Síntoma	Causa	Solución
`ModuleNotFoundError: No module named 'data_prep'` o `'nni'`	El notebook no se abrió desde el repo	Abre en VS Code la carpeta `curso-redes-neuronales` completa; en Colab, clona el repo y haz `%cd curso-redes-neuronales/notebooks`
`FileNotFoundError: data.csv`	Se ejecutó desde otra carpeta	Igual que arriba: el notebook debe correr dentro de `notebooks`
`TypeError: unsupported operand ... NoneType`	Un `TODO` sigue devolviendo `None`	Completa la función y vuelve a ejecutar desde esa celda
En `1_07` la pérdida no baja nada	Usaste `weights = weights + ...`	Usa `weights += ...` (ver C2)
La pérdida sube y baja (`WARNING - La pérdida ha aumentado`)	Tasa de aprendizaje muy grande	Baja `learnrate` (por ejemplo a 0.1)
La exactitud es 0.60 y no sube	Muy pocas épocas	Sube `epochs` a ~100
`ValueError: shapes not aligned`	`weights` con tamaño distinto al número de entradas	`size=n_features` en la inicialización
`1_08` da una recta distinta a la de practica_04	Datos sin escalar y pesos iniciales al azar	Es normal (ver B4)
Resultados distintos cada vez	Números al azar sin semilla	Ejecuta la celda con `np.random.seed(42)` antes